

Практичне заняття №4

DTO + class-validator + Pipes

Дисципліна: Високорівневі мови програмування та фреймворки

Мета

Додати валідацію вхідних даних до MiniShop API: створити DTO-класи (Data Transfer Objects) з декораторами class-validator, підключити глобальний ValidationPipe, замінити нетипізовані body: any та Partial<> на строго типізовані DTO в контролерах і сервісах. Створити кастомний Pipe для практичного розуміння механізму трансформації даних.

Передумови

Виконане Практичне заняття №3: MiniShop API з CRUD для categories та products працює через docker compose, міграції створюють таблиці, всі 10 ендпоінтів відповідають коректно.

Що здаємо (тільки одне)

Лінк на GitHub репозиторій, в якому є:

- README.md з коротким звітом (шаблон нижче) + виводом команд
- DTO-класи для categories та products у папках dto/
- Глобальний ValidationPipe у main.ts
- Контролери та сервіси оновлені для роботи з DTO
- Мінімум 3 нових коміти

Важливо: body: any та Partial<> в контролерах мають бути повністю замінені на DTO-класи.

Вправа 1 — Встановити пакети валідації

Опис

Встановити class-validator, class-transformer та @nestjs/mapped-types — три пакети, які разом забезпечують систему валідації в NestJS. class-validator надає декоратори (@IsString, @IsNumber тощо), class-transformer перетворює plain JSON у екземпляри класів, а @nestjs/mapped-types дає утиліти PartialType, PickType, OmitType для комбінування DTO.

Інструкції (покроково)

1) Встановити пакети через Docker:

```
docker compose run --rm app npm install class-validator class-transformer
@nestjs/mapped-types
```

2) Перевірити, що пакети додалися до package.json:

```
docker compose run --rm app sh -c "cat package.json | grep -E
\"class-validator|class-transformer|mapped-types\""
```

Очікується: три рядки з версіями пакетів.

 *class-validator та class-transformer працюють у парі. Без class-transformer ValidationPipe не зможе перетворити JSON-об'єкт у екземпляр DTO-класу, і декоратори валідації не спрацюють.*

Очікуваний результат

- package.json містить class-validator, class-transformer, @nestjs/mapped-types
- Застосунок запускається без помилок

Вправа 2 — Підключити глобальний ValidationPipe

Опис

Зареєструвати ValidationPipe глобально в main.ts. Глобальна реєстрація означає, що кожен вхідний запит до кожного контролера автоматично проходить через валідацію — не потрібно додавати Pipe до кожного методу окремо.

Інструкції (покроково)

1) Відкрити файл src/main.ts та оновити його:

```
import { NestFactory } from '@nestjs/core';
import { ValidationPipe } from '@nestjs/common';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
```

```

        transform: true,
      })),
    );

    await app.listen(3000);
  }
  bootstrap();

```

2) Перезапустити застосунок:

```

docker compose restart app
docker compose logs -f app

```

Пояснення опцій ValidationPipe

Опція	Що робить
whitelist: true	Видаляє з тіла запиту всі поля, яких немає в DTO. Клієнт надіслав isAdmin: true — це поле зникне.
forbidNonWhitelisted: true	Повертає 400 Bad Request, якщо в тілі є зайві поля. Суворіше за whitelist — клієнт одразу дізнається про помилку.
transform: true	Автоматично перетворює plain JSON-об'єкт у екземпляр DTO-класу. Також конвертує типи: рядок "42" → число 42.

⚠ Без transform: true URL-параметри завжди залишаються рядками, навіть з ParseIntPipe. З transform: true типи конвертуються автоматично.

Очікуваний результат

- main.ts містить глобальний ValidationPipe з трьома опціями
- Застосунок перезапускається без помилок
- localhost:3000 все ще відповідає

Вправа 3 — Створити DTO для Categories

Опис

Створити `CreateCategoryDto` та `UpdateCategoryDto` — два класи, які описують форму вхідних даних для створення та оновлення категорій. DTO замінять нетипізовані `body`: `{ name: string; description?: string }` та `Partial<>` з Практичної 3.

Інструкції (покроково)

1) Створити папку `dto/` в модулі `categories`:

```
mkdir -p src/categories/dto
```

2) Створити файл `src/categories/dto/create-category.dto.ts`:

```
import { IsString, IsOptional, MaxLength, MinLength } from 'class-validator';
```

```
export class CreateCategoryDto {
  @IsString()
  @MinLength(2)
  @MaxLength(100)
  name: string;

  @IsOptional()
  @IsString()
  @MaxLength(500)
  description?: string;
}
```

Пояснення декораторів

Декоратор	Що перевіряє
<code>@IsString()</code>	Поле має бути рядком. Не числом, не масивом — рядком.
<code>@MinLength(2)</code>	Рядок не коротший за 2 символи. Категорія "А" — це помилка.
<code>@MaxLength(100)</code>	Рядок не довший за 100 символів.
<code>@IsOptional()</code>	Поле можна не передавати. Але якщо передали — решта декораторів застосовуються.

3) Створити файл `src/categories/dto/update-category.dto.ts`:

```
import { PartialType } from '@nestjs/mapped-types';
import { CreateCategoryDto } from '../create-category.dto';

export class UpdateCategoryDto extends PartialType(CreateCategoryDto) {}
```

💡 *PartialType бере всі поля з CreateCategoryDto і робить їх необов'язковими. Це DRY — Don't Repeat Yourself. Ви описуєте валідацію один раз і перевикористовуєте. Якщо завтра додасте нове поле в CreateCategoryDto — UpdateCategoryDto автоматично його підхопить.*

Очікуваний результат

- Файл src/categories/dto/create-category.dto.ts створений з декораторами валідації
- Файл src/categories/dto/update-category.dto.ts створений через PartialType

Вправа 4 — Створити DTO для Products

Опис

Створити CreateProductDto та UpdateProductDto з повною валідацією. Продукт складніший за категорію: має числові поля (price, stock), необов'язковий зв'язок з категорією (categoryId), і потребує валідації числових діапазонів.

Інструкції (покроково)

1) Створити папку dto/ в модулі products:

```
mkdir -p src/products/dto
```

2) Створити файл src/products/dto/create-product.dto.ts:

```
import {
  IsString,
  IsNumber,
  IsOptional,
  IsInt,
  Min,
  MaxLength,
  MinLength,
} from 'class-validator';

export class CreateProductDto {
  @IsString()
  @MinLength(2)
  @MaxLength(255)
  name: string;

  @IsNumber({ maxDecimalPlaces: 2 })
  @Min(0.01)
  price: number;
```

```

@IsOptional()
@IsString()
@MaxLength(1000)
description?: string;

@IsOptional()
@IsInt()
@Min(0)
stock?: number;

@IsOptional()
@IsInt()
categoryId?: number;
}

```

Пояснення декораторів

Декоратор	Що перевіряє
@IsNumber({ maxDecimalPlaces: 2 })	Число з максимум 2 десятковими знаками. 999.99 — валідно, 9.999 — ні.
@Min(0.01)	Число не менше 0.01. Ціна 0 або від'ємна — помилка.
@IsInt()	Тільки цілі числа. 50 — валідно, 50.5 — ні.
@Min(0)	Число не менше 0. Stock не може бути від'ємним.

3) Створити файл src/products/dto/update-product.dto.ts:

```

import { PartialType } from '@nestjs/mapped-types';
import { CreateProductDto } from '../create-product.dto';

export class UpdateProductDto extends PartialType(CreateProductDto) {}

```

 **Зверніть увагу:** `categoryId` в DTO — це просте число, а не об'єкт `Category`. Клієнт надсилає `categoryId: 1`, а сервіс сам знаходить категорію. DTO описує HTTP-контракт, а не структуру бази даних.

Очікуваний результат

- Файл `src/products/dto/create-product.dto.ts` створений з повною валідацією

- Файл `src/products/dto/update-product.dto.ts` створений через `PartialType`

Вправа 5 — Оновити контролер та сервіс `Categories`

Опис

Замінити нетипізовані `body: { name: string; description?: string }` та `Partial<>` на DTO-класи в контролері та сервісі категорій. Це головна зміна — тепер типи визначаються в одному місці (DTO), а не дублюються в кожному методі.

Інструкції (покроково)

1) Оновити `src/categories/categories.controller.ts`:

```
import {
  Controller,
  Get,
  Post,
  Patch,
  Delete,
  Param,
  Body,
  ParseIntPipe,
} from '@nestjs/common';
import { CategoriesService } from '../categories.service';
import { CreateCategoryDto } from '../dto/create-category.dto';
import { UpdateCategoryDto } from '../dto/update-category.dto';

@Controller('api/categories')
export class CategoriesController {
  constructor(
    private readonly categoriesService: CategoriesService,
  ) {}

  @Get()
  findAll() {
    return this.categoriesService.findAll();
  }

  @Get('/:id')
  findOne(@Param('id', ParseIntPipe) id: number) {
    return this.categoriesService.findOne(id);
  }
}
```

```

@Post()
create(@Body() dto: CreateCategoryDto) {
    return this.categoriesService.create(dto);
}

@Patch('/:id')
update(
    @Param('id', ParseIntPipe) id: number,
    @Body() dto: UpdateCategoryDto,
) {
    return this.categoriesService.update(id, dto);
}

@Delete('/:id')
remove(@Param('id', ParseIntPipe) id: number) {
    return this.categoriesService.remove(id);
}
}

```

Що змінилось: замість `@Body() body: { name: string; description?: string }` тепер `@Body() dto: CreateCategoryDto`. Контролер більше не описує структуру даних — він посилається на DTO.

2) Оновити `src/categories/categories.service.ts`:

```

import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Category } from './category.entity';
import { CreateCategoryDto } from './dto/create-category.dto';
import { UpdateCategoryDto } from './dto/update-category.dto';

@Injectable()
export class CategoriesService {
    constructor(
        @InjectRepository(Category)
        private readonly categoryRepo: Repository<Category>,
    ) {}

    async findAll(): Promise<Category[]> {
        return this.categoryRepo.find();
    }

    async findOne(id: number): Promise<Category> {
        const category = await this.categoryRepo.findOne({

```



```

        where: { id },
    });
    if (!category) {
        throw new NotFoundException(
            `Category #${id} not found`,
        );
    }
    return category;
}

async create(dto: CreateCategoryDto): Promise<Category> {
    const category = this.categoryRepo.create(dto);
    return this.categoryRepo.save(category);
}

async update(
    id: number,
    dto: UpdateCategoryDto,
): Promise<Category> {
    const category = await this.findOne(id);
    Object.assign(category, dto);
    return this.categoryRepo.save(category);
}

async remove(id: number): Promise<void> {
    const category = await this.findOne(id);
    await this.categoryRepo.remove(category);
}
}

```

Що змінилось у сервісі: параметр `data: Partial<Category>` замінений на конкретний DTO-тип. Тепер TypeScript точно знає, які поля може містити об'єкт. Спроба передати `dto.id` або `dto.createdAt` — помилка компіляції.

Очікуваний результат

- Контролер `categories` використовує `CreateCategoryDto` та `UpdateCategoryDto`
- Сервіс `categories` приймає DTO замість `Partial<Category>`
- `body: any` та `Partial<>` повністю відсутні в коді `categories`

Вправа 6 — Оновити контролер та сервіс Products

Опис

Аналогічно оновити контролер та сервіс продуктів. Продукти складніші через зв'язок з категоріями — сервіс приймає `categoryId` (число) і сам знаходить відповідну категорію.

Інструкції (покроково)

1) Оновити `src/products/products.controller.ts`:

```
import {
  Controller,
  Get,
  Post,
  Patch,
  Delete,
  Param,
  Body,
  ParseIntPipe,
} from '@nestjs/common';
import { ProductsService } from '../products.service';
import { CreateProductDto } from '../dto/create-product.dto';
import { UpdateProductDto } from '../dto/update-product.dto';

@Controller('api/products')
export class ProductsController {
  constructor(
    private readonly productsService: ProductsService,
  ) {}

  @Get()
  findAll() {
    return this.productsService.findAll();
  }

  @Get('/:id')
  findOne(@Param('id', ParseIntPipe) id: number) {
    return this.productsService.findOne(id);
  }

  @Post()
  create(@Body() dto: CreateProductDto) {
    return this.productsService.create(dto);
  }

  @Patch('/:id')
  update(
```

```

        @Param('id', ParseIntPipe) id: number,
        @Body() dto: UpdateProductDto,
    ) {
        return this.productsService.update(id, dto);
    }

    @Delete('/:id')
    remove(@Param('id', ParseIntPipe) id: number) {
        return this.productsService.remove(id);
    }
}

```

2) Оновити src/products/products.service.ts:

```

import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Product } from '../product.entity';
import { CreateProductDto } from '../dto/create-product.dto';
import { UpdateProductDto } from '../dto/update-product.dto';

@Injectable()
export class ProductsService {
    constructor(
        @InjectRepository(Product)
        private readonly productRepo: Repository<Product>,
    ) {}

    async findAll(): Promise<Product[]> {
        return this.productRepo.find({
            relations: ['category'],
        });
    }

    async findOne(id: number): Promise<Product> {
        const product = await this.productRepo.findOne({
            where: { id },
            relations: ['category'],
        });
        if (!product) {
            throw new NotFoundException(
                `Product #${id} not found`,
            );
        }
        return product;
    }
}

```

```

async create(dto: CreateProductDto): Promise<Product> {
    const product = this.productRepo.create({
        name: dto.name,
        description: dto.description,
        price: dto.price,
        stock: dto.stock ?? 0,
        category: dto.categoryId
        ? { id: dto.categoryId } as any
        : undefined,
    });
    return this.productRepo.save(product);
}

async update(
    id: number,
    dto: UpdateProductDto,
): Promise<Product> {
    const product = await this.findOne(id);

    if (dto.name !== undefined) product.name = dto.name;
    if (dto.description !== undefined)
        product.description = dto.description;
    if (dto.price !== undefined) product.price = dto.price;
    if (dto.stock !== undefined) product.stock = dto.stock;
    if (dto.categoryId !== undefined) {
        product.category = { id: dto.categoryId } as any;
    }

    return this.productRepo.save(product);
}

async remove(id: number): Promise<void> {
    const product = await this.findOne(id);
    await this.productRepo.remove(product);
}
}

```

💡 **Зверніть увагу на метод `update`: замість `Object.assign(product, dto)` ми перевіряємо кожне поле окремо. Чому? Тому що `Object.assign` з `PartialType` може записати `undefined` поверх існуючих значень. Поле-за-полем оновлення — безпечніший підхід.**

3) Перезапустити застосунок та перевірити логи:

```

docker compose restart app
docker compose logs -f app

```

Очікується: 10 маршрутів зареєстровано без помилок.

Очікуваний результат

- Контролер products використовує CreateProductDto та UpdateProductDto
- Сервіс products приймає DTO, а не Partial<> або inline-типи
- Застосунок запускається, 10 маршрутів зареєстровано

Вправа 7 — Тестування валідації через curl

Опис

Перевірити, що валідація працює коректно: валідні дані проходять, невалідні — повертають 400 Bad Request з описом помилок. Також перевірити, що зайві поля відхиляються (forbidNonWhitelisted).

Тести валідації Categories

1. Валідне створення категорії (має працювати як раніше):

```
curl -X POST http://localhost:3000/api/categories \
  -H "Content-Type: application/json" \
  -d '{"name": "Electronics", "description": "Gadgets and devices"}'
```

Очікується: 201 Created, JSON з id, name, description, createdAt.

2. Порожнє ім'я — валідація name:

```
curl -X POST http://localhost:3000/api/categories \
  -H "Content-Type: application/json" \
  -d '{"name": ""}'
```

Очікується: 400 Bad Request з повідомленням про мінімальну довжину name.

3. Без поля name — обов'язкове поле:

```
curl -X POST http://localhost:3000/api/categories \
  -H "Content-Type: application/json" \
  -d '{"description": "Some text"}'
```

Очікується: 400 Bad Request з повідомленням, що name обов'язковий.

4. Зайве поле — forbidNonWhitelisted:

```
curl -X POST http://localhost:3000/api/categories \
  -H "Content-Type: application/json" \
  -d '{"name": "Test", "isAdmin": true}'
```

Очікується: 400 Bad Request з повідомленням "property isAdmin should not exist".

Тести валідації Products

5. Валідне створення продукту:

```
curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "iPhone 16", "price": 999.99, "stock": 50, "categoryId": 1}'
```

Очікується: 201 Created.

6. Від'ємна ціна:

```
curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Bad Product", "price": -5}'
```

Очікується: 400 Bad Request, "price must not be less than 0.01".

7. Декілька помилок одразу:

```
curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "", "price": -5, "stock": -10}'
```

Очікується: 400 Bad Request з масивом BCIX помилок (name, price, stock).

Валідатор збирає всі помилки, а не зупиняється на першій.

8. Валідне оновлення (PATCH — часткові дані):

```
curl -X PATCH http://localhost:3000/api/products/1 \
  -H "Content-Type: application/json" \
  -d '{"price": 899.99}'
```

Очікується: 200 OK, оновлений продукт. Завдяки PartialType можна надіслати тільки одне поле.

9. Порожнє тіло PATCH:

```
curl -X PATCH http://localhost:3000/api/products/1 \
  -H "Content-Type: application/json" \
  -d '{}'
```

Очікується: 200 OK (порожнє оновлення — валідне, бо всі поля необов'язкові в UpdateProductDto).

💡 *Windows без curl: використайте Postman або PowerShell Invoke-RestMethod, як на Практичній 3.*

Вправа 8 — Створити кастомний Pipe (TrimPipe)

Опис

Створити кастомний Pipe, який автоматично обрізає пробіли (trim) з рядкових полів тіла запиту. Це типова задача: користувач може випадково додати пробіл на початку або в кінці назви — " iPhone 16 " має стати "iPhone 16". Pipe вирішує це автоматично для всіх ендпоінтів.

Інструкції (покроково)

1) Створити файл src/common/pipes/trim.pipe.ts:

```
mkdir -p src/common/pipes
```

```
import { PipeTransform, Injectable, ArgumentMetadata } from '@nestjs/common';

@Injectable()
export class TrimPipe implements PipeTransform {
  transform(value: any, metadata: ArgumentMetadata) {
    // Застосовуємо тільки до body
    if (metadata.type !== 'body' || typeof value !== 'object') {
      return value;
    }

    const trimmed: Record<string, any> = {};
    for (const [key, val] of Object.entries(value)) {
      trimmed[key] = typeof val === 'string'
        ? val.trim()
        : val;
    }
    return trimmed;
  }
}
```

Пояснення

Кожен Pipe реалізує інтерфейс `PipeTransform` з одним методом `transform(value, metadata)`. `value` — це вхідні дані (тіло запиту, параметр URL, query). `metadata` — інформація про тип (body, param, query) та очікуваний клас. Pipe повертає оброблене значення — або кидає виняток, якщо дані невалідні.

2) Зареєструвати TrimPipe глобально в main.ts (ПЕРЕД ValidationPipe):

```
import { NestFactory } from '@nestjs/core';
import { ValidationPipe } from '@nestjs/common';
import { AppModule } from './app.module';
import { TrimPipe } from './common/pipes/trim.pipe';

async function bootstrap() {
```

```
const app = await NestFactory.create(AppModule);

app.useGlobalPipes(
  new TrimPipe(),
  new ValidationPipe({
    whitelist: true,
    forbidNonWhitelisted: true,
    transform: true,
  }),
);

await app.listen(3000);
}
bootstrap();
```

⚠️ Порядок Pipes має значення! TrimPipe має стояти ПЕРЕД ValidationPipe. Спочатку обрізаємо пробіли, потім валідуємо. Якщо навпаки — рядок " " пройде MinLength(2), але потім обріжеться до "".

3) Перезапустити та протестувати:

```
docker compose restart app
```

```
curl -X POST http://localhost:3000/api/categories \
  -H "Content-Type: application/json" \
  -d '{"name": " Accessories  "'}
```

Очікується: категорія створена з name "Accessories" (без пробілів на початку та в кінці).

```
curl -X POST http://localhost:3000/api/categories \
  -H "Content-Type: application/json" \
  -d '{"name": "  "'}
```

Очікується: 400 Bad Request. TrimPipe обріже пробіли, і MinLength(2) відхилить порожній рядок.

Очікуваний результат

- Файл src/common/pipes/trim.pipe.ts створений
- TrimPipe зареєстрований глобально перед ValidationPipe
- Рядки автоматично очищуються від зайвих пробілів

Вправа 9 — Оновити README та зробити коміти

Інструкції (покроково)

1) Переконайтесь, що .env додано до .gitignore.

2) Зробити коміти (мінімум 3):

```
git add package.json package-lock.json src/main.ts
git commit -m "add class-validator, configure global ValidationPipe"
```

```
git add src/categories/dto/ src/products/dto/
git commit -m "add DTOs with validation for categories and products"
```

```
git add src/categories/categories.controller.ts
src/categories/categories.service.ts
git add src/products/products.controller.ts src/products/products.service.ts
git add src/common/
git commit -m "use DTOs in controllers/services, add TrimPipe"
```

```
git add README.md
git commit -m "update readme for practical 4"
```

```
git push
```

3) Оновити README.md за шаблоном:

```
## Student
```

```
- Name: <ПІБ>
```

```
- Group: <Група>
```

```
## Практичне заняття №4 – DTO + class-validator + Pipes
```

```
### Структура репозиторію
```

```
```
```

```
.
├── src/
│ ├── categories/
│ │ ├── dto/
│ │ │ ├── create-category.dto.ts
│ │ │ └── update-category.dto.ts
│ │ ├── category.entity.ts
│ │ ├── categories.module.ts
│ │ ├── categories.service.ts
│ │ └── categories.controller.ts
│ ├── products/
│ │ ├── dto/
│ │ │ ├── create-product.dto.ts
│ │ │ └── update-product.dto.ts
│ │ └── product.entity.ts
```

```
| | | └─ products.module.ts
| | | └─ products.service.ts
| | | └─ products.controller.ts
| | └─ common/
| | | └─ pipes/
| | | | └─ trim.pipe.ts
| | └─ migrations/
| | └─ data-source.ts
| | └─ main.ts
| | └─ app.module.ts
└─ Dockerfile
└─ docker-compose.yml
└─ README.md
...

```

### Запуск проекту

```
```bash
cp .env.example .env
docker compose up --build
...

```

Тест валідації – порожнє ім'я категорії

```
```text
<вивід curl POST /api/categories з {"name": ""}>
...

```

### Тест валідації – від'ємна ціна продукту

```
```text
<вивід curl POST /api/products з {"name": "Test", "price": -5}>
...

```

Тест валідації – зайве поле

```
```text
<вивід curl POST /api/categories з {"name": "Test", "isAdmin": true}>
...

```

### Тест TrimPipe

```
```text
<вивід curl POST /api/categories з {"name": " Trimmed "}>
...

```

Тест валідне створення продукту

```
```text
<вивід curl POST /api/products з валідними даними>
...

```

## Troubleshooting — типові проблеми

### 1) "An unknown value was passed to the validate function"

Симптоми: помилка при виклику POST/PATCH ендпоінтів.

#### Що зробити:

- Переконайтесь, що class-transformer встановлений: `docker compose run --rm app npm ls class-transformer`
- Перевірити, що `transform: true` увімкнено в `ValidationPipe`
- Перезібрати образ: `docker compose up --build`

### 2) Валідація не працює — будь-які дані проходять

Симптоми: POST з порожнім тілом або невалідними даними повертає 201 замість 400.

#### Що зробити:

- Перевірити, що `ValidationPipe` додано в `main.ts`:  
`app.useGlobalPipes(...)`
- Перевірити, що контролер використовує DTO-тип: `@Body() dto: CreateProductDto` (не `body: any`)
- Перевірити, що у `tsconfig.json` є: `"emitDecoratorMetadata": true` та `"experimentalDecorators": true`

### 3) "Cannot find module './dto/create-category.dto'"

Симптоми: помилка при компіляції TypeScript.

#### Що зробити:

- Перевірити шлях: файл має бути в `src/categories/dto/create-category.dto.ts`
- Перевірити правильність імпорту в контролері та сервісі
- Перевірити, що ім'я файлу точно збігається (case-sensitive в Linux)

### 4) "property isAdmin should not exist" — але я не надсилав isAdmin

Симптоми: 400 помилка з несподіваним полем.

#### Що зробити:

- Перевірити JSON в curl — можлива помилка в лапках або структурі
- Спробувати через Postman — буде видно точне тіло запиту

- Якщо потрібно тимчасово дозволити зайві поля — змінити `forbidNonWhitelisted` на `false` (але для задачі має бути `true`)

#### **5) TrimPipe не працює — пробіли залишаються**

Симптоми: `name` зберігається з пробілами.

##### **Що зробити:**

- Перевірити порядок `Pipes` в `main.ts`: `TrimPipe` має бути ПЕРЕД `ValidationPipe`
- Перевірити, що `TrimPipe` імпортований правильно
- Перезапустити: `docker compose restart app`